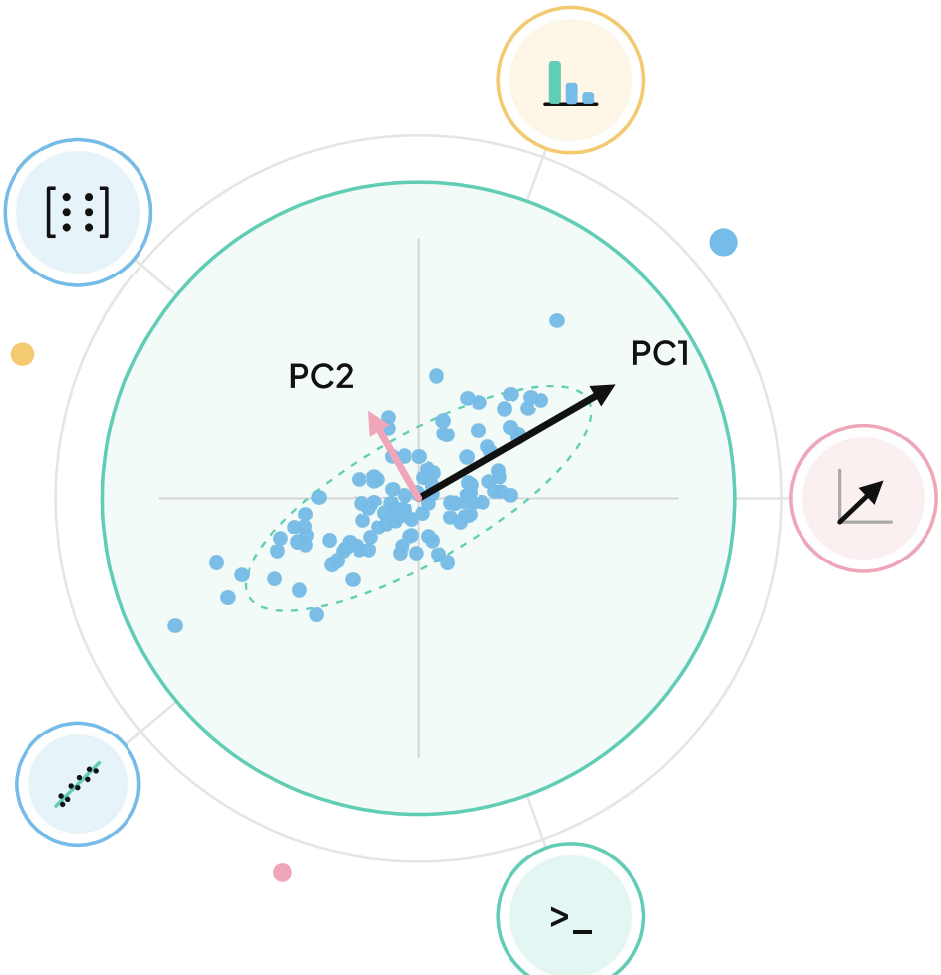
Aula 6.11 · Módulo 6 — Machine Learning na Mão, com NumPy

PCA POR ÁLGEBRA LINEAR

Como transformar quatro dimensões em duas utilizando matemática e Python.

- dados originais
- centralização
- direções principais
- projeção
- redução



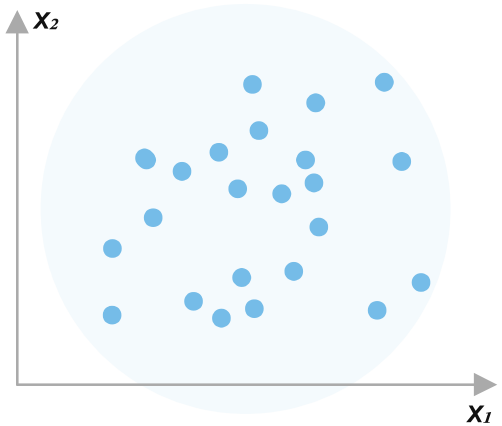
Mapa da aula · 120 minutos

10 min	20 min	25 min	15 min	15 min	30 min	5 min
Redução de dimensio-nalidade	**Centralização variância, covariância**	**Autovalores e autovetores**	**Componentes e projeção**	**SVD e relação com PCA**	**Prática: Iris 4D → 2D**	**Quiz e desafio**

PROGRESSÃO Intuição → Álgebra Linear → Estatística → PCA → NumPy → SVD → Scikit-learn

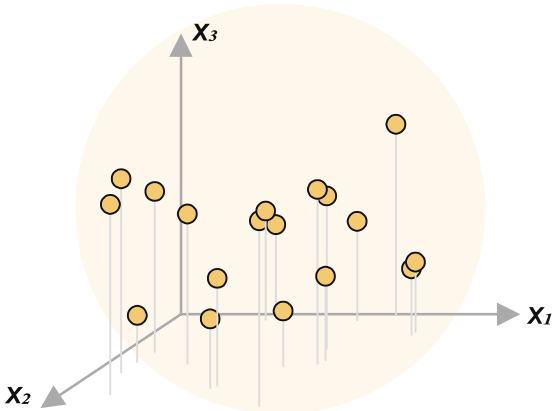
É possível representar um conjunto de dados utilizando menos características sem perder toda a informação?

Características e dimensões



p = 2

Cada amostra é um ponto no plano. Um eixo por característica.



p = 3

Um ponto no espaço. Ainda desenhável, em perspectiva.

$$(x_1, x_2, \ldots, xp)$$

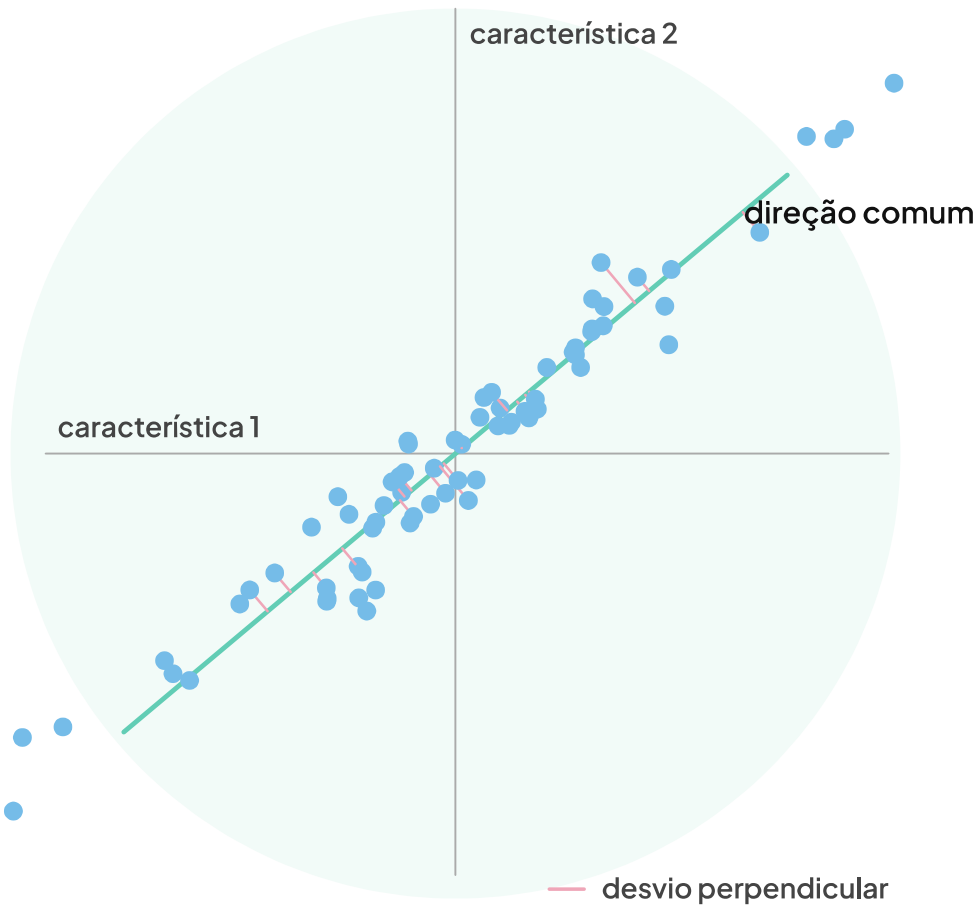
p ≥ 4

Uma lista de p números. Não há gráfico cartesiano comum com todos os eixos.

Redundância entre características

Quando duas características variam juntas, parte do que uma informa já está na outra. A nuvem se alonga em uma direção.

compressão	descrever cada ponto pela posição ao longo da diagonal: 2 números → 1
custo	o desvio perpendicular à diagonal é descartado
regra geral	reduzir dimensões normalmente implica perder alguma informação



Análise de Componentes Principais

Principal Component Analysis procura novas direções *ortogonais* nas quais os dados apresentam a *maior variância possível*, em ordem decrescente.

---

componente principal
vetor unitário que define uma nova direção em $\mathbb{R}^p$

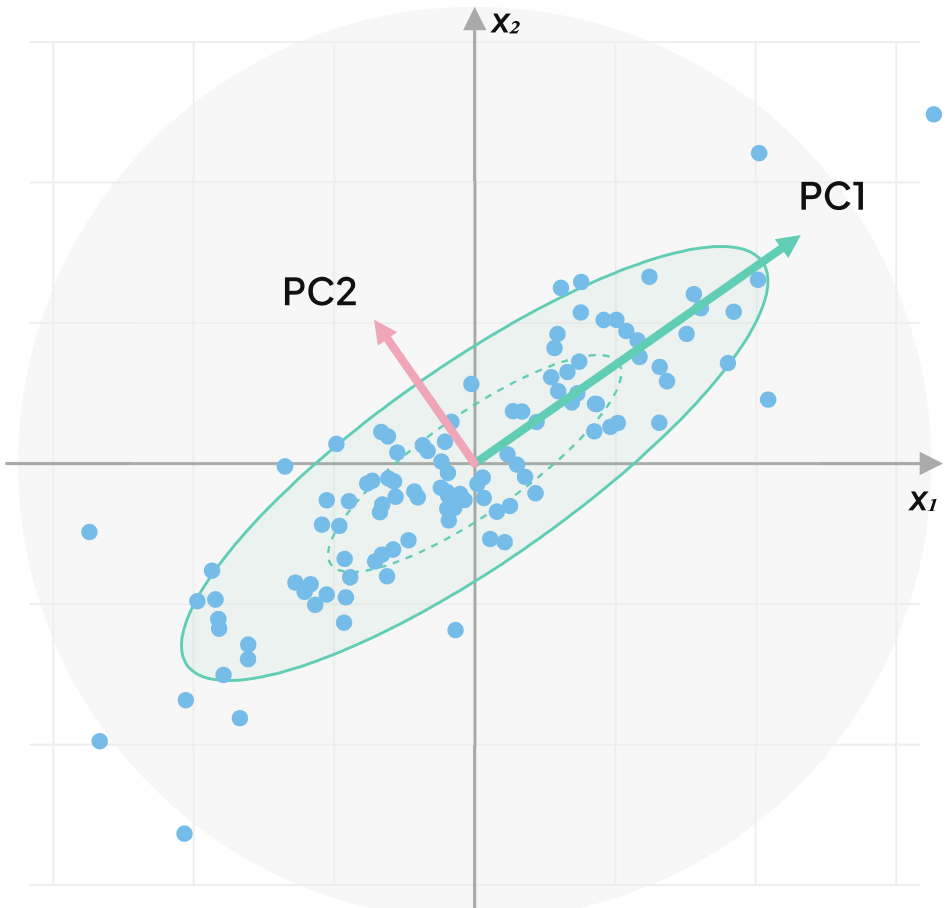
direção
reta pela origem; v e –v indicam a mesma direção

variância
espalhamento médio ao quadrado em torno da média

projeção
coordenada de um ponto ao longo de uma direção

ortogonalidade
vetores perpendiculares: produto escalar igual a zero

transformação linear
multiplicação por uma matriz; aqui, uma rotação dos eixos



A direção de maior espalhamento

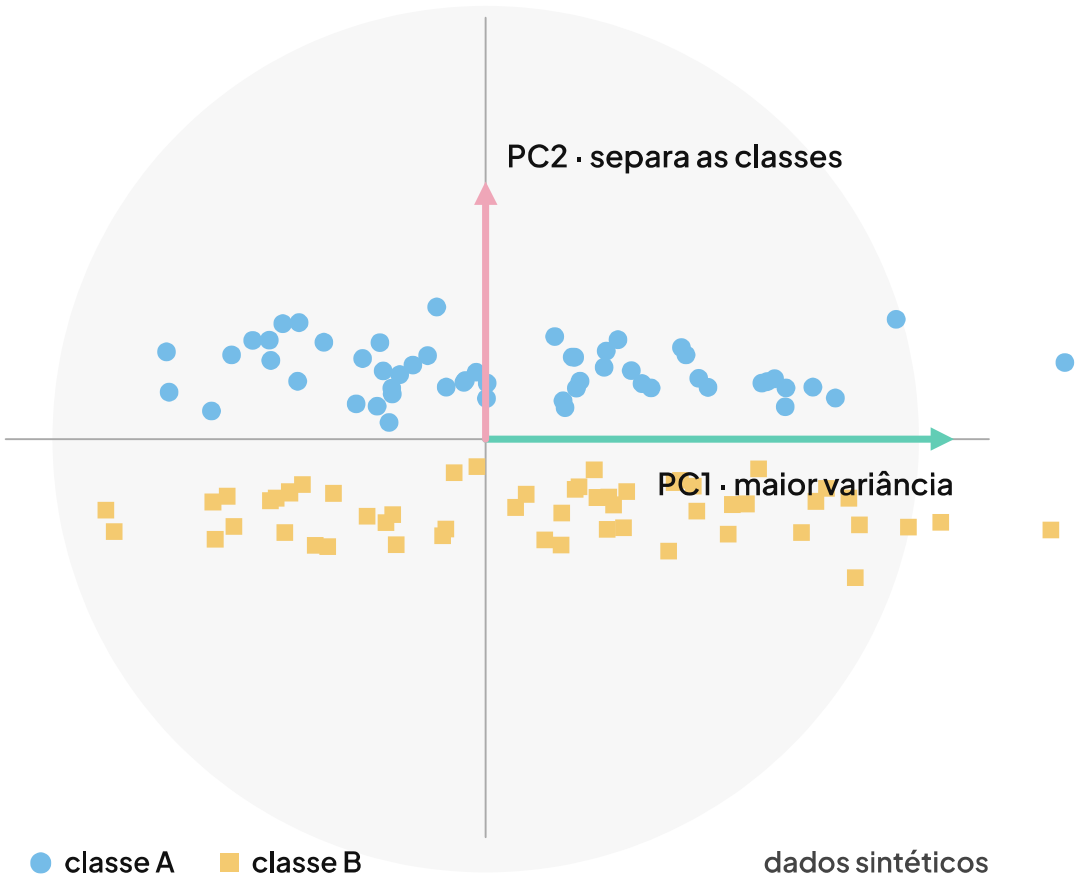
- Eixos originais x_1 e x_2: espalhamento parecido em cada um.
- PC1: direção de maior variância da nuvem.
- PC2: perpendicular a PC1, com a variância restante.

PC1 e PC2 são uma rotação dos eixos em torno da média. Dados sintéticos.

Variância elevada não garante relevância

As classes se separam ao longo de PC2, a direção de menor variância. Manter apenas PC1 mistura as classes.

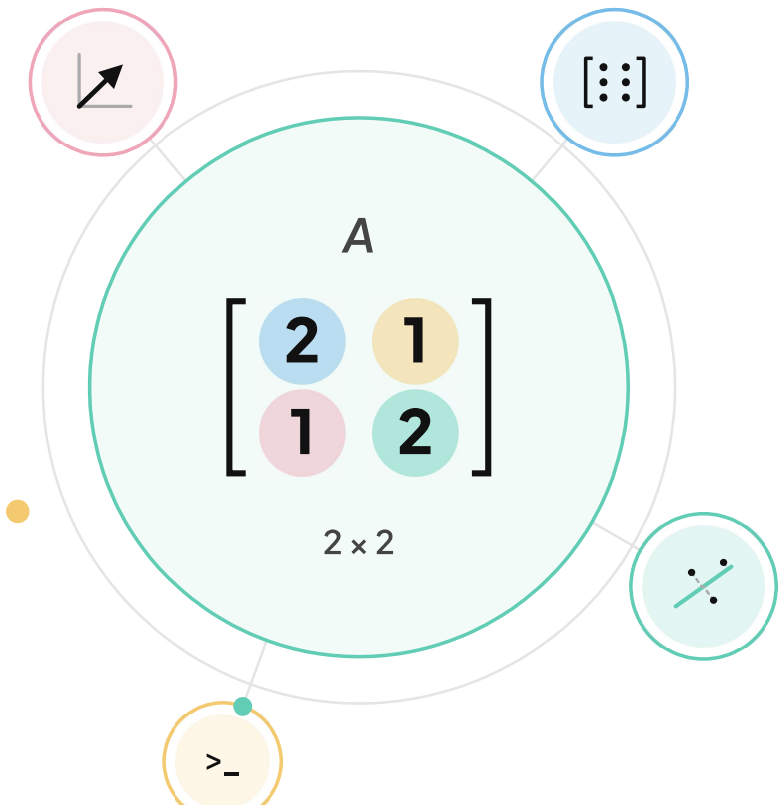
PCA é não supervisionado: maximiza variância sem consultar rótulos.



03

Álgebra Linear

a linguagem das direções



Escalares, vetores e a matriz de dados

$$X \in \mathbb{R}^{n \times p}$$

	col 1	col 2
linha 1	1	1
linha 2	2	2
linha 3	3	3

Linha destacada: a amostra 1. Contorno: $x_{31} = 3$.

escalar	um número: 3,5
vetor	lista ordenada de números: [2, 2]
n, p	n amostras (linhas), p características (colunas)
x_{ij}	valor da característica j na amostra i
X^T	transposta: troca linhas por colunas, formato p×n

Produto escalar, norma e ortogonalidade

produto escalar

$$u \cdot v = \Sigma_i \, u_i \, v_i$$

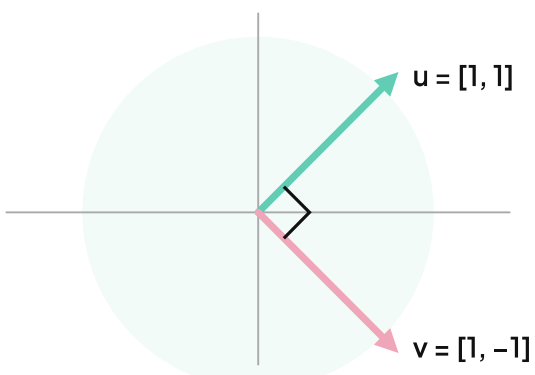
```
u = [1, 1]
v = [1, -1]
u·v = 1·1 + 1·(-1) = 0
```

norma

$$\|u\| = \sqrt{(u \cdot u)}$$

```
‖u‖ = √(1² + 1²) = √2
u/‖u‖ = [1/√2, 1/√2]
norma 1: vetor unitário
```

ortogonalidade



$u \cdot v = 0 \Leftrightarrow u \perp v$. Se $\|w\| = 1$, $x \cdot w$ é a coordenada de x ao longo de w.

Produto matricial e formatos

$$(AB)_{ij} = \Sigma_k \, a_{ik} \, b_{kj}$$

Cada elemento do resultado é o produto escalar de uma linha de A com uma coluna de B.

$$\begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 3 & 3 \end{bmatrix} @ \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

linha 1: 1·1 + 1·1 = 2

regra dos formatos

$$(n \times p)(p \times k) = n \times k$$

(3×2)(2×1) = 3×1

covariância: (p×n)(n×p) = p×p

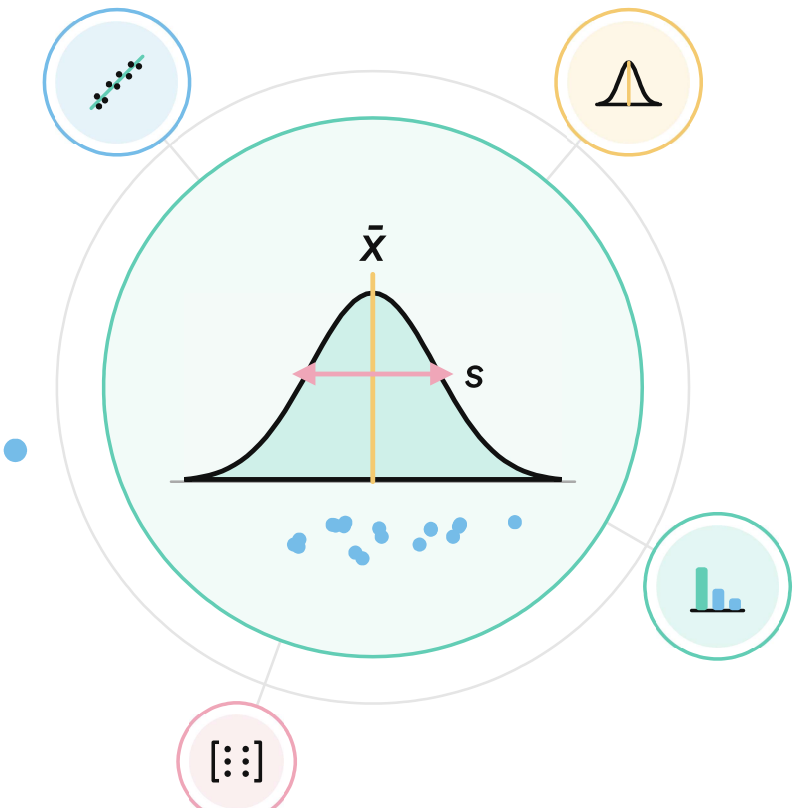
projeção: (n×p)(p×k) = n×k

NumPy: A @ B é produto matricial; A * B é elemento a elemento

04

Estatística

média, variância, covariância

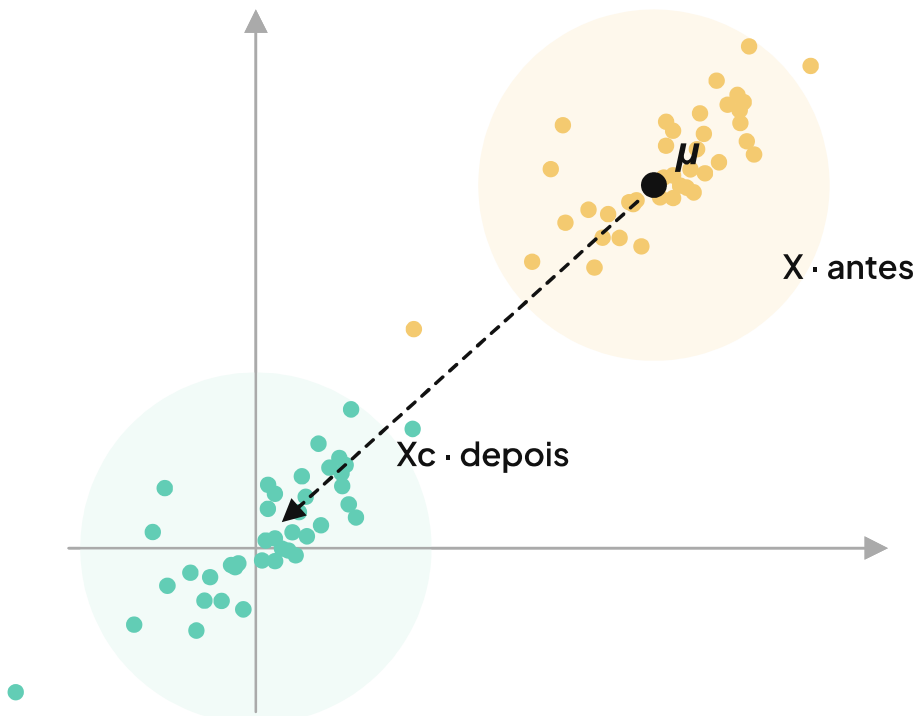


Média de cada característica

$$\mu_j = (1/n)\,\Sigma_{i=1}^{n}\, x_{ij}$$

$$X_c = X - \mu$$

μ_j	média da característica (coluna) j
x_{ij}	valor da característica j na amostra i
n	número de amostras; i percorre 1, …, n
μ	vetor das p médias, subtraído de cada linha
X_c	dados centralizados: cada coluna com média 0



Centralizar translada a nuvem até a origem. A forma não muda.

Centralização passo a passo

1 · dados

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 3 & 3 \end{bmatrix}$$

2 · média por coluna

$\mu_1 = (1+2+3)/3 = 2$

$\mu_2 = (1+2+3)/3 = 2$

$\mu = [2, 2]$

3 · subtrair μ de cada linha

$[1-2, 1-2] = [-1, -1]$

$[2-2, 2-2] = [\ 0, 0]$

$[3-2, 3-2] = [\ 1, 1]$

$$X_c = \begin{bmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

Soma de cada coluna: −1 + 0 + 1 = 0. As médias de X_c são [0, 0].

Centralização em NumPy

```python
import numpy as np

X = np.array([[1, 1], [2, 2], [3, 3]],
             dtype=float)

# axis=0: desce pelas linhas → uma média por coluna
media = X.mean(axis=0)

# broadcasting: (3, 2) - (2,) → (3, 2)
X_centralizado = X - media

print(media, media.shape)
print(X_centralizado)
print(X_centralizado.mean(axis=0))
```

SAÍDA

```
[2. 2.] (2,)
[[-1. -1.]
 [ 0.  0.]
 [ 1.  1.]]
[0. 0.]
```

`axis=0` → média de cada característica, formato (p,)

`axis=1` → média de cada amostra, formato (n,): erro neste contexto

Broadcasting repete `media` para cada linha.

Variância amostral

$$s^2 = \Sigma_i (x_i - \bar{x})^2 / (n - 1)$$

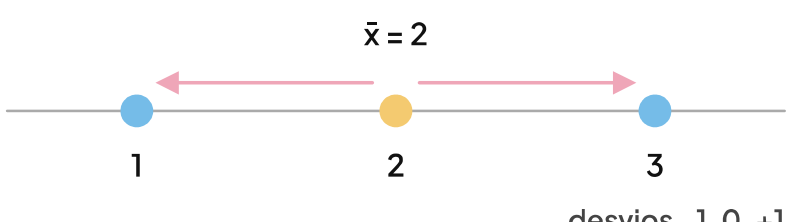
s^2	variância amostral
$x_i - \bar{x}$	desvio da amostra i em relação à média $\bar{x}$
$( )^2$	evita cancelamento e pesa mais desvios grandes
$n - 1$	amostral (não viesada); a populacional σ^2 divide por n

np.var(x) usa ddof=0 · np.var(x, ddof=1) e np.cov usam n − 1

exemplo · x = [1, 2, 3]

x_i	$x_i - \bar{x}$	$(x_i - \bar{x})^2$
1	-1	1
2	0	0
3	1	1
$\bar{x} = 2$	$\Sigma = 0$	$\Sigma = 2$

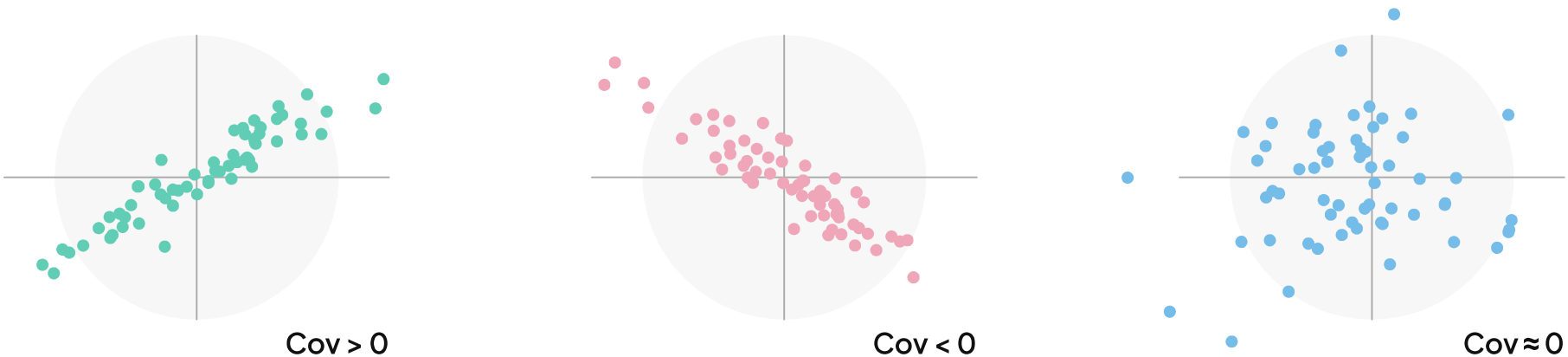
$s^2 = 2 / (3 - 1) = 1$



Covariância

$$Cov(x, y) = \Sigma_i (x_i - \bar{x})(y_i - \bar{y}) / (n - 1)$$

Mede como duas variáveis variam conjuntamente. $\bar{x}$ e $\bar{y}$ são as médias; n, o número de amostras; $Cov(x, x) = s^2_x$.



exemplo · x = y = [1, 2, 3]

`(-1)(-1) + (0)(0) + (1)(1) = 2 → 2/2 = 1`

cuidado · x = [−1, 0, 1], y = x²

Cov = 0, mas y depende de x. Covariância zero não implica independência.

Matriz de covariância

$$C = X_c^T X_c / (n - 1)$$

X_c	dados centralizados, n×p
X_c^T	transposta, p×n
C	(p×n)(n×p) = p×p
C_{jk}	$\Sigma_i (x_{ij} - \mu_j)(x_{ik} - \mu_k) / (n - 1)$
$C^T = C$	simétrica: $C_{jk} = C_{kj}$



Diagonal (destaque): variâncias. Fora dela: covariâncias. Exemplo com p = 3.

Produto $X_c^T X_c$ passo a passo

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} @ \begin{bmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \div (3-1) = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

(2×3) (3×2) (2×2)

```
(1,1): (-1)(-1) + 0·0 + 1·1 = 2
(1,2): (-1)(-1) + 0·0 + 1·1 = 2
(2,1) = (2,2) = 2 · colunas iguais
```

Diagonal: $s^2_1 = s^2_2 = 1$.

Fora da diagonal: $\text{Cov}(x_1, x_2) = 1$, positiva.

Simétrica: $C_{12} = C_{21}$.

Covariância em NumPy

```
# fórmula: (p×n)(n×p) / (n - 1) → p×p
n = len(X)
cov_manual = (X_centralizado.T @ X_centralizado
              / (n - 1))

# rowvar=False: cada COLUNA é uma variável
cov_numpy = np.cov(X_centralizado, rowvar=False)

print(cov_manual)
print(cov_manual.shape)
print(np.allclose(cov_manual, cov_numpy))
```

SAÍDA

```
[[1. 1.]
 [1. 1.]]
(2, 2)
True
```

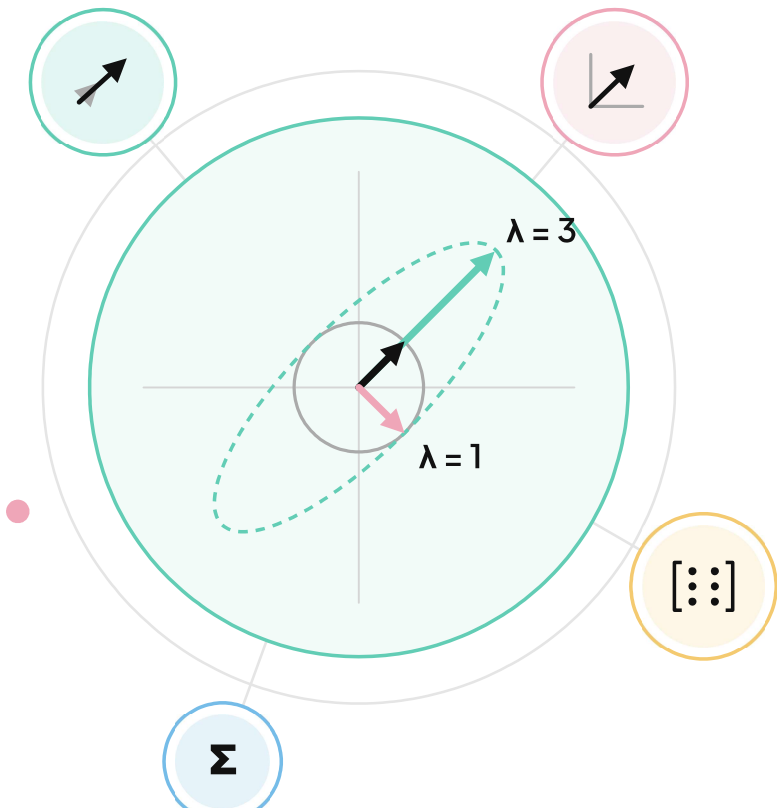
Sem `rowvar=False`, np.cov trataria cada linha como variável e devolveria 3×3.

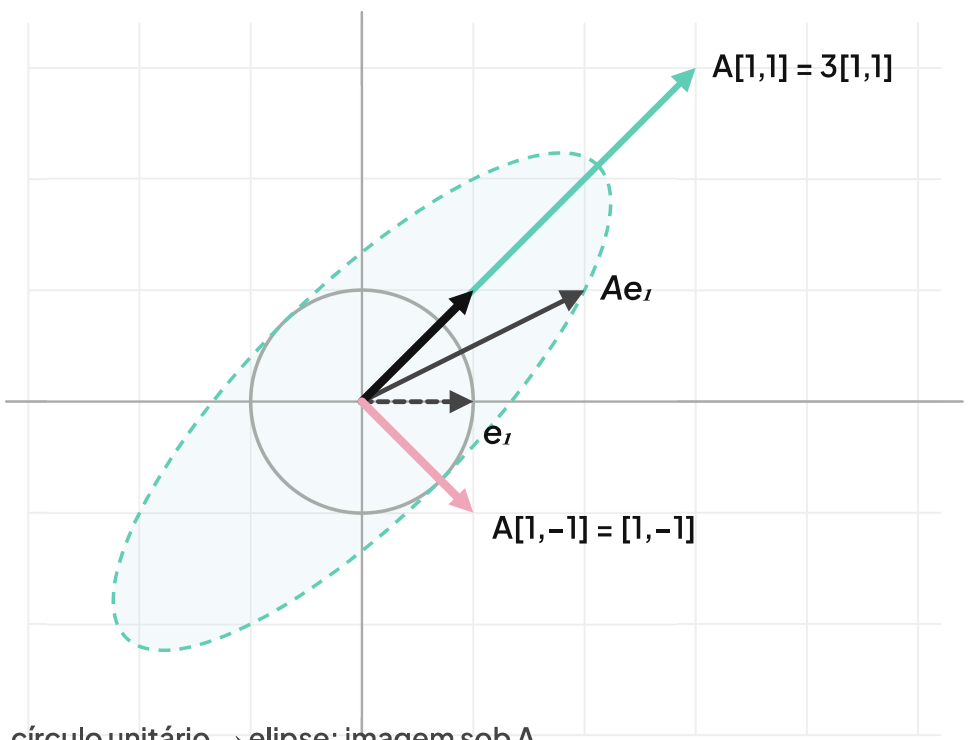
`np.allclose` compara com tolerância numérica.

08

Autovalores e autovetores

direções que a matriz apenas escala





círculo unitário → elipse: imagem sob A

Direções que permanecem na mesma reta

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

```
A [1, 0] = [2, 1]   · saiu da reta
A [1, 1] = [3, 3] = 3·[1, 1]
A [1, -1] = [1, -1] = 1·[1, -1]
```

Autovetor: direção mantida pela transformação.
Autovalor: fator de escala nessa direção.

Autovetor e autovalor

$$Av = \lambda v$$

A	matriz quadrada p×p (a transformação)
v	autovetor: vetor não nulo, a direção preservada
λ	autovalor: escalar que estica, comprime ou inverte v
Av	produto matricial · λv: escalonamento

verificação manual · A = [[2, 1], [1, 2]]

```
v = [1, 1]
Av = [2·1 + 1·1, 1·1 + 2·1]
   = [3, 3] = 3·v
```

`λ = 3`

```
v = [1, -1]
Av = [2 - 1, 1 - 2] = [1, -1] = 1·v
```

`λ = 1`

Qualquer múltiplo não nulo de v, inclusive –v, também satisfaz Av = λv.

Autovetores da covariância são as direções principais

Variância dos dados projetados em uma direção unitária w:

$$Var(X_c w) = w^T X_c^T X_c w / (n-1) = w^T C w$$

Se w é autovetor de C, então $Cw = \lambda w$ e

$$w^T C w = \lambda\, w^T w = \lambda$$

O máximo de $w^T Cw$ com $\|w\| = 1$ é o maior autovalor, atingido no seu autovetor.

autovetor	direção principal (componente)
autovalor	variância dos dados ao longo dessa direção
C simétrica	autovalores reais e ≥ 0; autovetores ortogonais
PC2, PC3…	maximizam $w^T Cw$ entre direções ortogonais às anteriores

Autovalores de C, passo a passo

$$Cv = \lambda v$$

$$(C - \lambda I)v = 0,\ v \neq 0$$

$$\det(C - \lambda I) = 0$$

I	identidade: 1 na diagonal, 0 fora; $Iv = v$
$\det$	determinante; para [[a, b], [c, d]]: $ad - bc$
λ	incógnita: o autovalor procurado
$\det = 0$	equação característica: $C - \lambda I$ não invertível, logo existe $v \neq 0$

```
C = [[1, 1], [1, 1]]

C - λI = [[1-λ, 1], [1, 1-λ]]

det = (1-λ)(1-λ) - 1·1

    = 1 - 2λ + λ² - 1

    = λ² - 2λ = λ(λ - 2) = 0
```

$$\lambda_1 = 2 \quad \lambda_2 = 0$$

conferência: $\lambda_1 + \lambda_2 = \text{traço}(C) = 2 \cdot \lambda_1\lambda_2 = \det(C) = 0$

Autovetores de C, passo a passo

$\lambda_1 = 2$

$(C - 2I) = [[-1, 1], [1, -1]]$

$-a + b = 0 \rightarrow b = a$

$v = [1, 1], \|v\| = \sqrt{2}$

$$v_1 = (1/\sqrt{2})[1, 1]$$

$\approx [0.7071, 0.7071]$

$\lambda_2 = 0$

$(C - 0I) = [[1, 1], [1, 1]]$

$a + b = 0 \rightarrow b = -a$

$v = [1, -1], \|v\| = \sqrt{2}$

$$v_2 = (1/\sqrt{2})[1, -1]$$

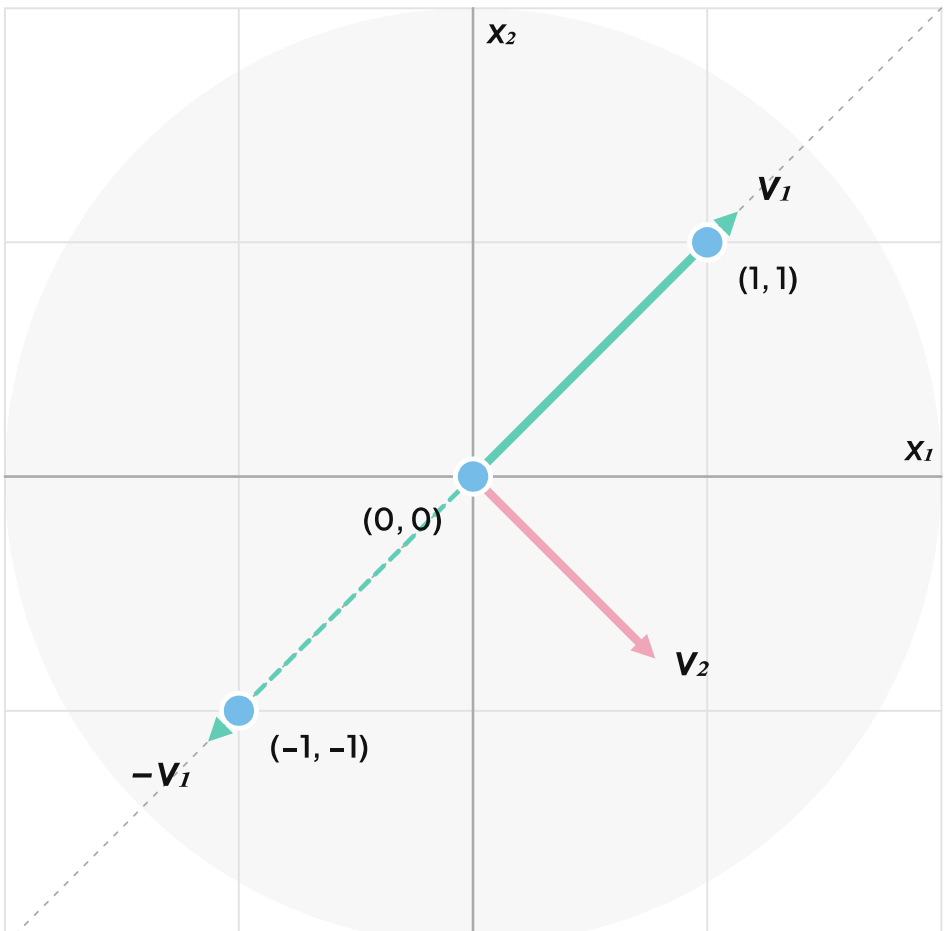
$\approx [0.7071, -0.7071]$

Ortogonais: $v_1 \cdot v_2 = \frac{1}{2}(1 \cdot 1 + 1 \cdot (-1)) = 0$.

Sinal: $-v_1$ também é autovetor unitário de $\lambda_1 = 2$ e indica a mesma reta.

Direções principais sobre os dados

- v_1: direção principal. Projeções $-\sqrt{2}, 0, \sqrt{2}$; variância 2.
- v_2: direção secundária, ortogonal. Projeções 0, 0, 0; variância 0.
- $-v_1$: mesma reta, sentido oposto. Mesmo componente.



Autovalores e autovetores com eigh

```
# eigh: matriz simétrica → valores reais,
# vetores ortonormais, ordem CRESCENTE
autovalores, autovetores = np.linalg.eigh(cov_manual)

print(autovalores.shape, autovetores.shape)
print(np.round(autovalores, 6))
print(np.round(autovetores, 4))

# cada COLUNA é um autovetor: C v = λ v
v = autovetores[:, 1]
print(np.allclose(cov_manual @ v, autovalores[1] * v))
```

```
SAÍDA (sinais podem variar)
(2,) (2, 2)
[0. 2.]
[[-0.7071  0.7071]
 [ 0.7071  0.7071]]
True
```

O autovalor 0 pode aparecer como ±1e-16: arredondamento.

Coluna i ↔ autovalor i. Não use a linha.

Ordenação dos componentes

```
# índices em ordem decrescente
indices = np.argsort(autovalores)[::-1]

autovalores = autovalores[indices]
# reordenar COLUNAS, não linhas
autovetores = autovetores[:, indices]
```

`argsort`: índices que ordenariam de forma crescente → [0, 1]

`[::-1]`: inverte → [1, 0]

`[:, indices]`: cada coluna acompanha seu autovalor

antes (eigh)

λ = 0	λ = 2
[-0.71, 0.71]	[0.71, 0.71]

depois · PC1 primeiro

λ = 2	λ = 0
[0.71, 0.71]	[-0.71, 0.71]

Variância explicada

$$EVR_k = \lambda_k / \Sigma_j \lambda_j$$

EVR_k	explained variance ratio do componente k
λ_k	variância ao longo do componente k
$\Sigma \lambda_j$	variância total = traço(C) = soma das variâncias
$\times 100$	proporção em porcentagem

exemplo manual · λ = [2, 0]

$EVR_1 = 2/2 = 100\%$

$EVR_2 = 0/2 = 0\%$

Os pontos estão sobre uma reta: uma coordenada basta.

contraexemplo · C = [[2, 1], [1, 2]] · λ = [3, 1]

$EVR_1 = 3/4 = 75\%$

$EVR_2 = 1/4 = 25\%$

Manter só PC1 perde 25% da variância.

Projeção dos dados

$$Z = X_c W$$

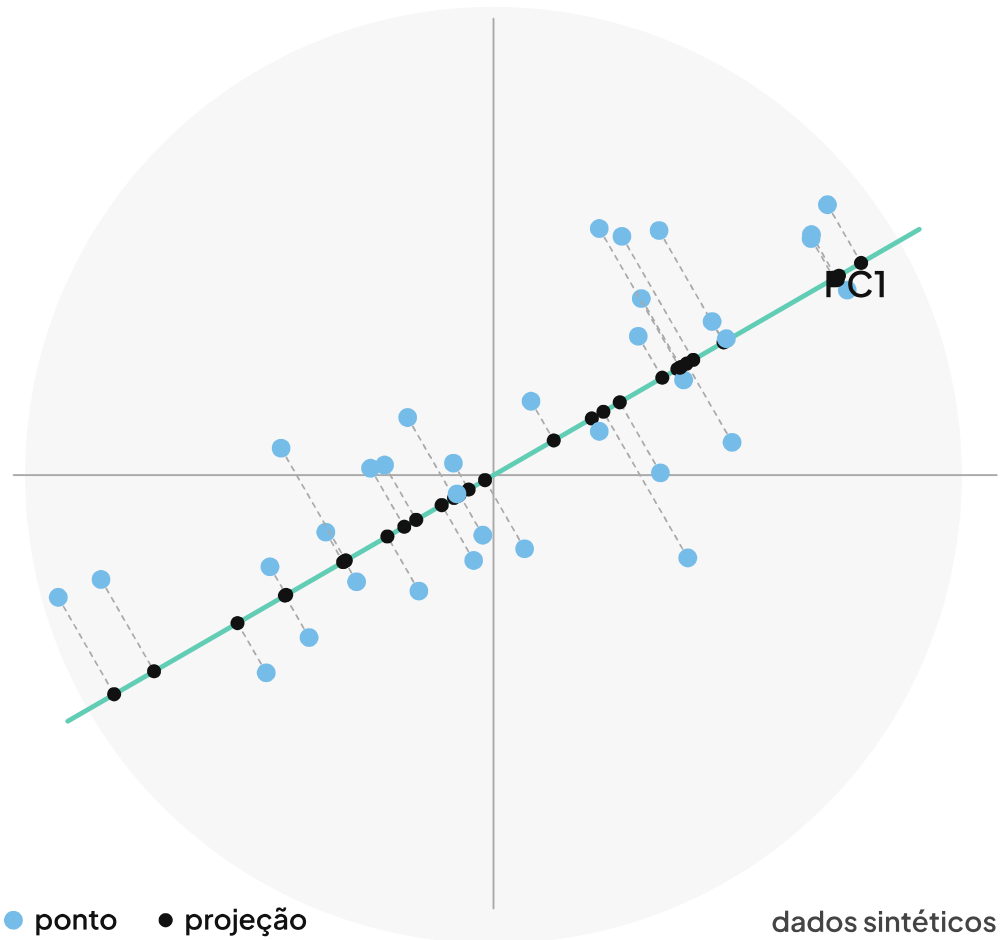
X_c	dados centralizados, n×p
W	k primeiros autovetores como colunas, p×k
Z	novas coordenadas (scores), n×k
z_{ik}	$x_{c,i} \cdot w_k$: coordenada da amostra i no componente k

exemplo · 2D → 1D · W = v_1 (2×1)

$$\begin{bmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix} @ \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix} = \begin{bmatrix} -1.414 \\ 0 \\ 1.414 \end{bmatrix}$$

$(3\times2)(2\times1) = 3\times1 \cdot Z = [-\sqrt{2}, 0, \sqrt{2}]$

$\mathrm{Var}(Z) = (2 + 0 + 2)/(3 - 1) = 2 = \lambda_1$



Projetar não é apagar colunas

Cada ponto desce perpendicularmente até PC1. Sua posição ao longo da reta é o score z.

PCA constrói novas características como combinações lineares das originais:

$$z = w_1 x_{c1} + w_2 x_{c2} + \ldots + w_p x_{cp}$$

Apagar x_2 equivale a projetar no eixo horizontal, que em geral não é a direção de maior variância.

Reconstrução dos dados

$$\hat{X} = ZW^T + \mu$$

$\hat{X}$	dados aproximados (reconstruídos), n×p
Z	projeção, n×k
W^T	transposta da matriz de componentes, k×p
μ	média original, devolve a nuvem à posição inicial

$W^TW = I_k$ porque as colunas de W são ortonormais.

exemplo · amostra 1 · z = -√2

$$z \cdot v_1^T = -\sqrt{2} \cdot (1/\sqrt{2})[1, 1]$$

$$= [-1, -1]$$

$$+ \mu = [-1, -1] + [2, 2] = [1, 1]$$

Igual à amostra original. Reconstrução exata porque $\lambda_2 = 0$: não havia variância fora de PC1.

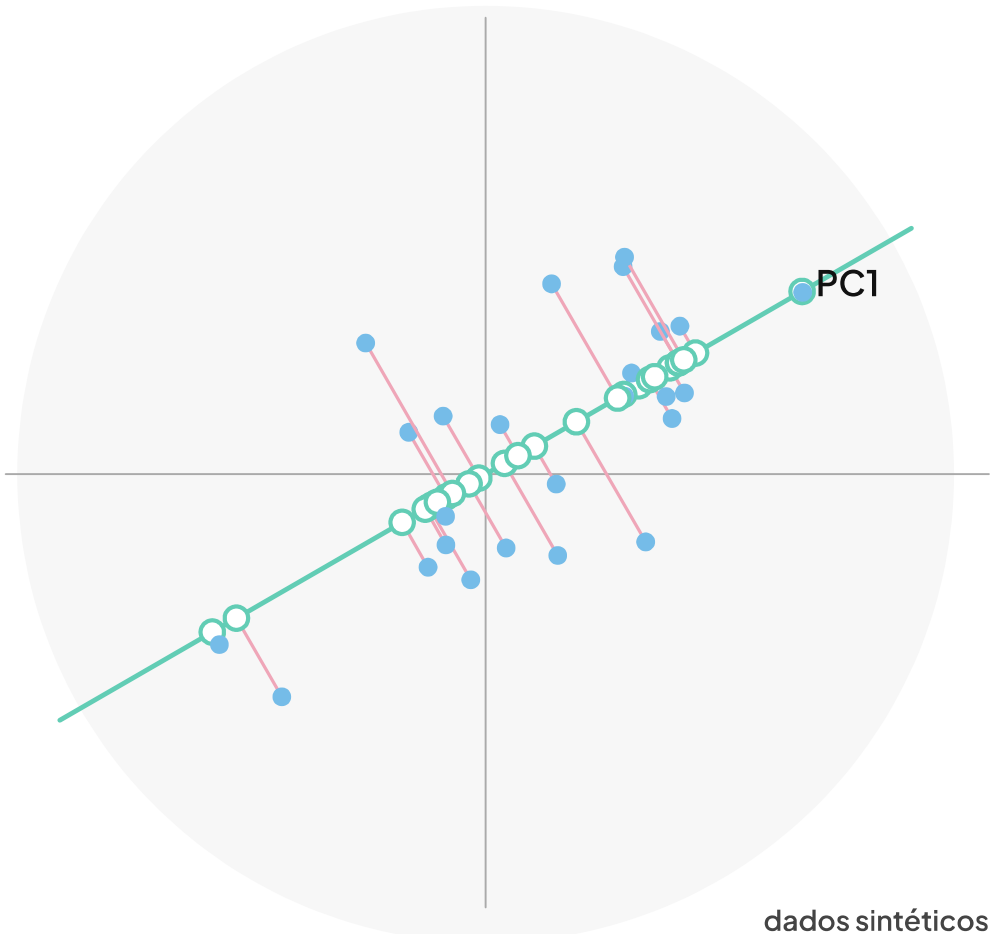
Erro de reconstrução

$$MSE = (1/np)\,\Sigma_{i,j}\,(x_{ij} - \hat{x}_{ij})^2$$

$\hat{x}_{ij}$: valor reconstruído; np: total de elementos.

Σ (resíduos)² = (n – 1) · Σ autovalores descartados.

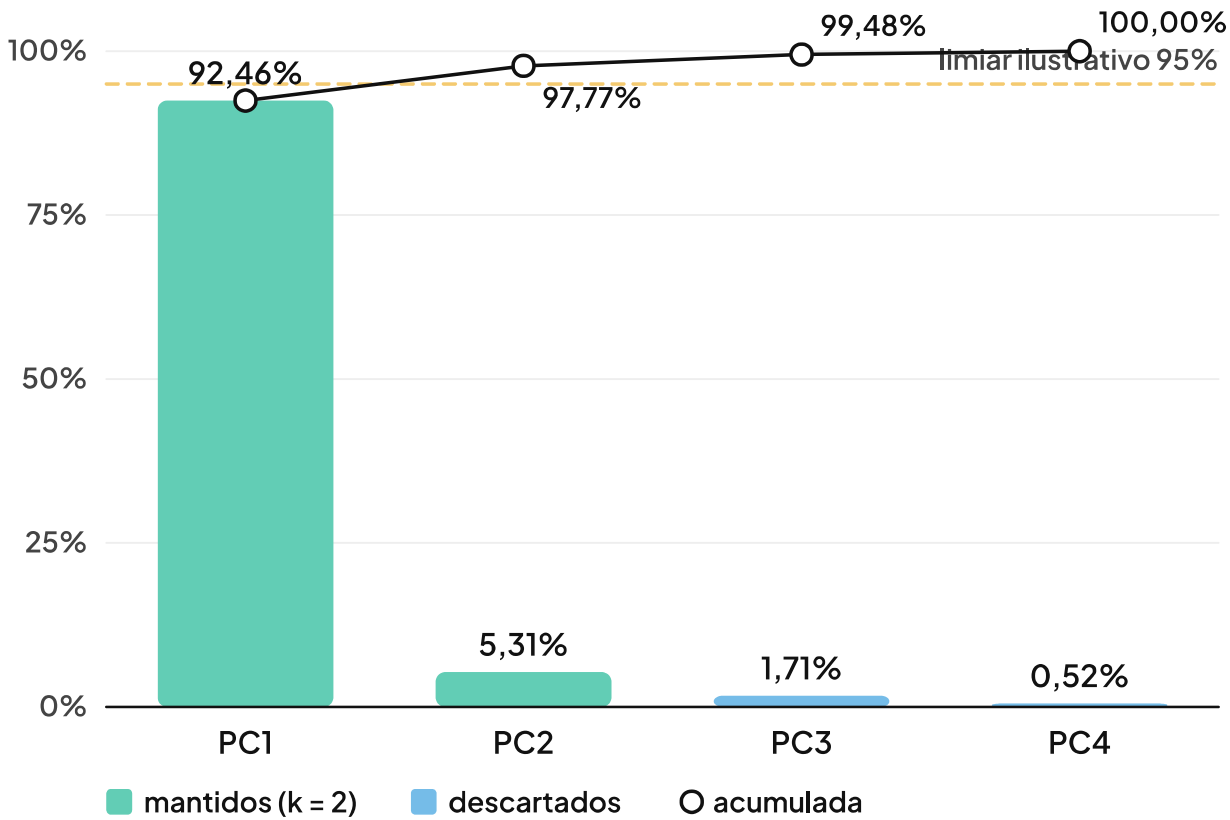
Iris com k = 2: MSE ≈ 0,0253.



dados sintéticos

● original ○ reconstruído (k = 1) dados sintéticos

Quantos componentes manter



menor k que atinge o limiar · Iris

≥ 90%	k	= 1
≥ 95%	k	= 2
≥ 99%	k	= 3

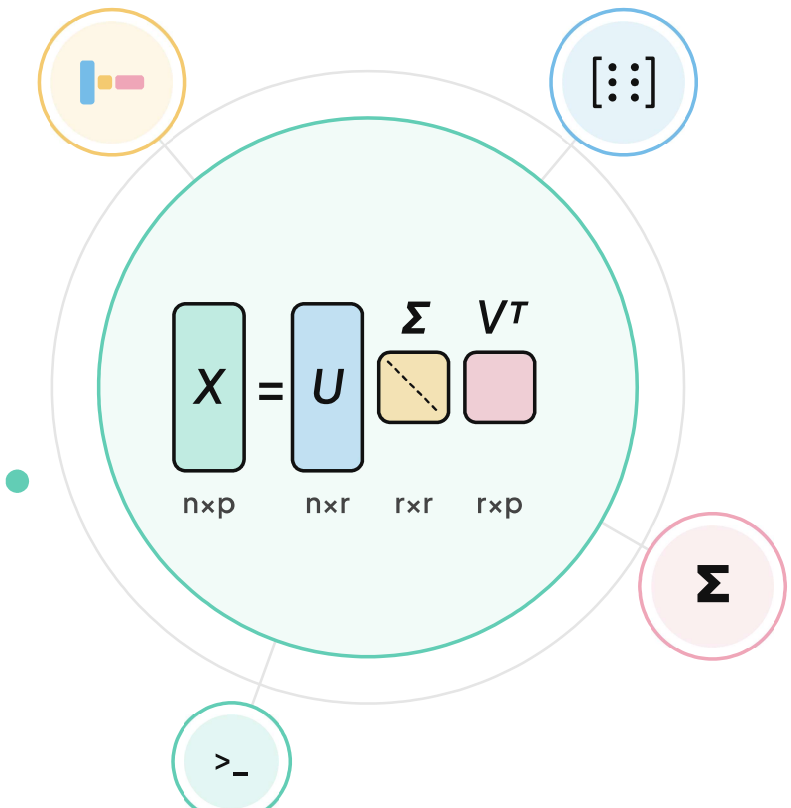
95% não é regra universal. É um compromisso entre compressão e informação preservada.

PCA sobre as características originais, apenas centralizadas.

16

SVD

decomposição em valores singulares



Decomposição em valores singulares

$$X_c = U\Sigma V^T$$

`(n×p) = (n×r)(r×r)(r×p) · r = min(n, p)`

U

n×r · vetores singulares à esquerda, colunas ortonormais: $U^TU = I$

Iris: 150×4

Σ

r×r diagonal · valores singulares $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$

Iris: 4×4 · NumPy devolve só a diagonal S

V^T

r×p · linhas = vetores singulares à direita, ortonormais: $V^TV = I$

Iris: 4×4

Geometria: V^T gira, Σ escala cada eixo, U posiciona no espaço das amostras. Exemplo manual: $\sigma_1 = 2$, $\sigma_2 = 0$.

Da SVD aos componentes principais

$$C = X_c^T X_c / (n-1)$$

$$= (U\Sigma V^T)^T (U\Sigma V^T) / (n-1)$$

$$= V\Sigma U^T U\Sigma V^T / (n-1)$$

$$= V\Sigma^2 V^T / (n-1) \quad U^T U = I$$

$$\lambda_k = \sigma_k^2 / (n-1)$$

V	colunas = autovetores de C = componentes principais
σ_k	valor singular; $\sigma_k^2 / (n-1)$ é o autovalor λ_k
Z	$X_c V = U\Sigma$: scores sem formar C
Iris	$25{,}10^2 \;/\; 149 \approx 4{,}228 = \lambda_1$

Vantagem numérica: evita formar $X_c^T X_c$, que eleva ao quadrado o número de condição.

PCA por SVD em NumPy

```
U, S, Vt = np.linalg.svd(
    X_centralizado,
    full_matrices=False
)
# S já vem em ordem decrescente
autovalores_svd = S**2 / (len(X) - 1)

# componentes = LINHAS de Vt
W_svd = Vt[:1].T          # p×k
Z_svd = X_centralizado @ W_svd

# comparar subespaços (sinal irrelevante)
W = autovetores[:, :1]
print(np.allclose(W @ W.T, W_svd @ W_svd.T))
```

```
FORMATOS E VALORES
U (3, 2) · S (2,) · Vt (2, 2)
S                ≈ [2. 0.]
autovalores_svd ≈ [2. 0.]
True
```

Use a mesma matriz centralizada nos dois métodos.

SVD de X sem centralizar não é PCA.

Sinais opostos, mesmo componente

`autovalores, EVR`	idênticos · comparar direto
`W` W^T	idêntico para W e −W · mesmo subespaço
$ZW^T + \mu$	idêntica: $(-z)(-w)^T = zw^T$
`colunas de Z`	podem trocar de sinal · gráfico espelhado

```
# alinhar o sinal de cada coluna de B a A
def alinhar_sinais(A, B):
    s = np.sign(np.sum(A * B, axis=0))
    s[s == 0] = 1
    return B * s

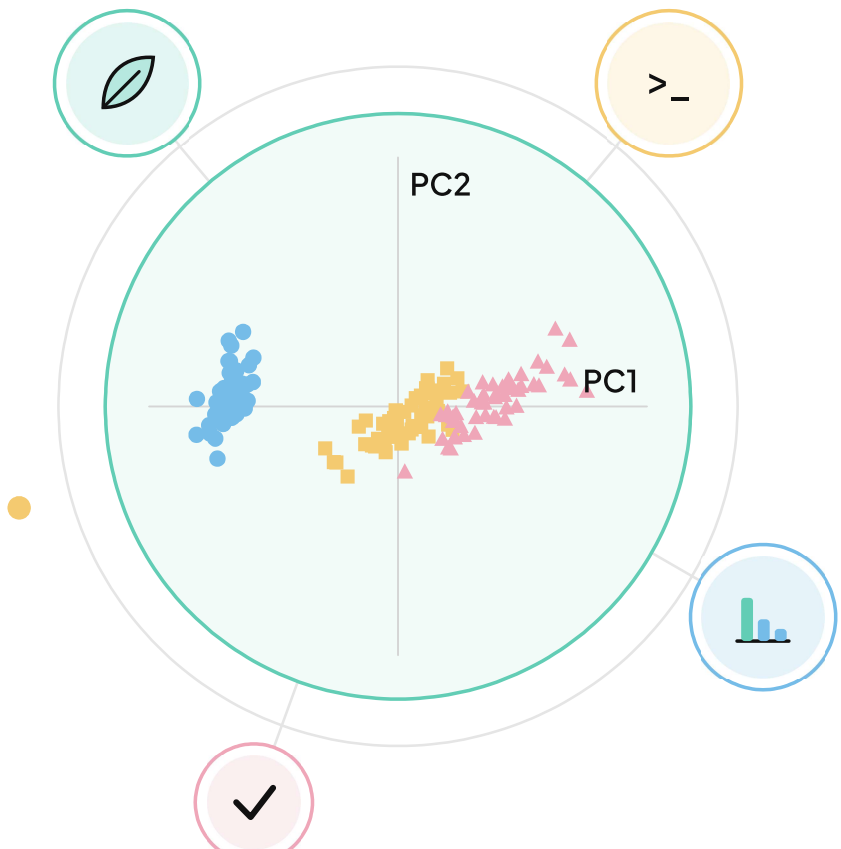
np.allclose(W, alinhar_sinais(W, W_svd))
np.allclose(Z, alinhar_sinais(Z, Z_svd))
```

Um componente com sinal invertido não é um componente diferente nem um erro.

prática

Comprimindo o Iris de 4D para 2D

do notebook vazio à comparação com o Scikit-learn



O exercício e o dataset

150	4	3	2
amostras (n)	características em cm (p)	espécies · 50 cada · só para cores	componentes mantidos (k)

`etapas 1–4`	`etapas 5–9`	`etapas 10–14`	`etapas 15–17`	`etapas 18–20`
imports, carregar, explorar, visualizar pares	média, centralizar, covariância, eigh, ordenar	variância explicada, W, projeção 150×2, gráfico	reconstrução, SVD, comparação de subespaços	Scikit-learn, comparação completa, conclusão

Convenção: medidas originais, apenas centralizadas, sem padronização adicional.

Dados e centralização

```
# 1 · imports
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
# 2 · carregar
iris = load_iris()
X = iris.data
y = iris.target   # só para colorir
# 3 · explorar
print(X.shape, iris.feature_names)
print(np.bincount(y), iris.target_names)
# 5 · média · 6 · centralizar
media = X.mean(axis=0)
X_centralizado = X - media
print(np.round(X_centralizado.mean(axis=0), 12))
```

```
SAÍDA
(150, 4) ['sepal length (cm)', …]
[50 50 50] ['setosa' 'versicolor'
            'virginica']
media ≈ [5.8433 3.0573 3.758  1.1993]
[ 0.  0.  0.  0.]   # ≈ 1e-15
```

Etapa 4 no notebook: 6 gráficos de pares. Nenhum mostra as 4 características juntas.

Pode aparecer –0. nas médias: é zero em ponto flutuante.

Covariância, autovalores e variância explicada

```
# 7 · matriz de covariância 4×4
n = len(X)
cov = X_centralizado.T @ X_centralizado / (n - 1)
print(np.allclose(cov, np.cov(X, rowvar=False)))
# 8 · autovalores e autovetores
autovalores, autovetores = np.linalg.eigh(cov)
# 9 · ordenar (decrescente) valores E colunas
indices = np.argsort(autovalores)[::-1]
autovalores = autovalores[indices]
autovetores = autovetores[:, indices]
# 10 · variância explicada · 11 · acumulada
evr = autovalores / autovalores.sum()
evr_acumulada = np.cumsum(evr)
```

```
COVARIÂNCIA (≈)
[[ 0.6857 -0.0424  1.2743  0.5163]
 [-0.0424  0.19   -0.3297 -0.1216]
 [ 1.2743 -0.3297  3.1163  1.2956]
 [ 0.5163 -0.1216  1.2956  0.581 ]]

AUTOVALORES ORDENADOS (≈)
[4.2282 0.2427 0.0782 0.0238]
```

Soma = traço(cov) ≈ 4,573: variância total.

Variância explicada do Iris

97,77%

da variância total preservada por PC1 + PC2.

Válido para o PCA sobre as características originais, apenas centralizadas. Com padronização, o valor muda (≈ 95,81%).

	λ	EVR	acumulada
PC1	4,2282	92,46%	92,46%
PC2	0,2427	5,31%	97,77%
PC3	0,0782	1,71%	99,48%
PC4	0,0238	0,52%	100,00%

Soma dos λ ≈ 4,5730. Valores verificados com os dados de load_iris().

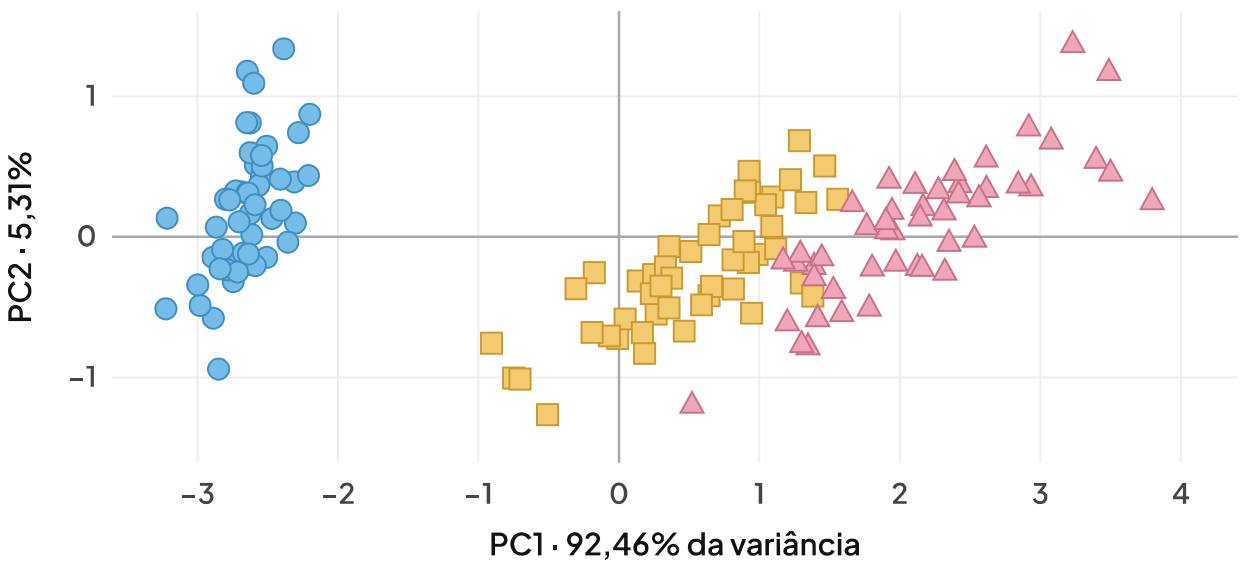
Projeção e reconstrução

```
# 12 · dois componentes principais
k = 2
W = autovetores[:, :k]            # (4, 2)
# 13 · projeção
Z = X_centralizado @ W            # (150, 2)
print(W.shape, Z.shape)
print(np.round(Z[:3], 4))
# 15 · reconstrução
X_reconstruido = Z @ W.T + media  # (150, 4)
mse = np.mean((X - X_reconstruido) ** 2)
sse = np.sum((X - X_reconstruido) ** 2)
print(mse)
print(np.isclose(sse, (n - 1) * autovalores[k:].sum()))
```

```
SAÍDA (sinais podem variar)
(4, 2) (150, 2)
[[-2.6841  0.3194]
 [-2.7141 -0.177 ]
 [-2.889  -0.1449]]
0.02534...
True
```

$(150\times4)(4\times2) = 150\times2 \cdot (150\times2)(2\times4) = 150\times4$

O Iris em duas dimensões



- setosa
- versicolor
- virginica

PCA ajustado **sem** os rótulos. As espécies entram só nas cores.

Projeção de 4 características em 2 eixos. Eixos na mesma escala. Um eixo espelhado é efeito de sinal.

SVD, Scikit-learn e comparação

```
# 16 · SVD da MESMA matriz centralizada
U, S, Vt = np.linalg.svd(X_centralizado,
                         full_matrices=False)
W_svd = Vt[:k].T
Z_svd = X_centralizado @ W_svd
# 18 · Scikit-learn
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
W_sk = pca.components_.T          # (4, 2)
# 19 · comparar sem depender do sinal
P = W @ W.T
print(np.allclose(P, W_svd @ W_svd.T),
      np.allclose(P, W_sk @ W_sk.T))
print(np.allclose(X_reconstruido,
      pca.inverse_transform(X_pca)))
```

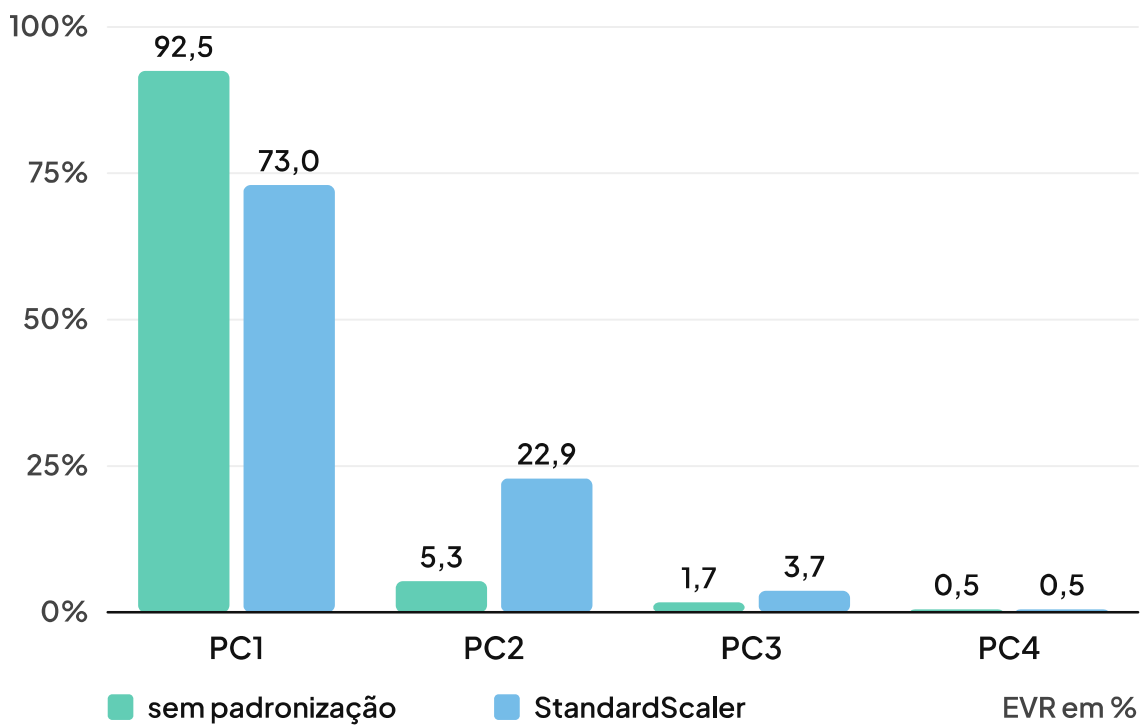
```
SAÍDA (≈)
S ≈ [25.1   6.0131 3.4137 1.8845]
S**2/(n-1) ≈ [4.2282 0.2427 …]
explained_variance_ratio_
  ≈ [0.9246 0.0531]
True True
True
```

`components_`: componentes nas linhas (k×p)

`explained_variance_`: autovalores, divisor n − 1

`inverse_transform`: $ZW^T + \mu$

Padronizar muda direções e variância explicada



sem padronização · covariância

PC1 + PC2 ≈ 97,77%. PC1 dominado pelo comprimento da pétala.

StandardScaler · correlação

PC1 + PC2 ≈ 95,81%. Pesos de PC1 mais equilibrados.

Análises diferentes, não erradas. Sempre declare qual foi feita.

PCA dentro de um experimento supervisionado

```python
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression

# 1 · separar ANTES de ajustar o PCA
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42,
    stratify=y)
# 2 · PCA ajustado só com o treino
modelo = make_pipeline(PCA(n_components=2),
                       LogisticRegression())
modelo.fit(X_tr, y_tr)
print(modelo.score(X_te, y_te))
```

exploração (exercício)

PCA no dataset inteiro para visualizar: adequado.

modelo supervisionado

μ e W são aprendidos: ajuste só no treino. Ajustar antes do split vaza o teste.

Erros comuns e como corrigi-los

conceito

PCA não seleciona colunas: cria combinações lineares.

PCA não preserva sempre 100%: verifique a EVR.

Média ≠ variância; variância ≠ covariância.

Autovalor é um número; autovetor é uma direção.

PCA não é supervisionado: não use y nos componentes.

cálculo

Esquecer de centralizar → confira $X_c.mean(axis=0) \approx 0$.

axis=1 na média → use axis=0, formato (p,).

Ordenar λ sem reordenar colunas → autovetores[:, indices].

Usar linha em vez de coluna → autovetores[:, i].

W com formato errado → W deve ser p×k; Z, n×k.

comparação

SVD de X sem centralizar → use X_centralizado.

Sinais opostos ≠ componentes diferentes → compare WW^T.

PCA antes do split → ajuste só no treino (Pipeline).

Padronizado vs. não padronizado → declare qual foi feito.

Igualdade exata em floats → use np.allclose.

Quiz

01 · O que é redução de dimensionalidade?

02 · O que significa PCA?

03 · O que é um componente principal?

04 · O que é centralização?

05 · O que representa uma matriz de covariância?

06 · Qual a diferença entre variância e covariância?

07 · O que é um autovetor?

08 · O que é um autovalor?

09 · Por que ordenamos os autovalores?

10 · O que significa variância explicada?

11 · Como projetamos os dados?

12 · O que significa reduzir de 4D para 2D?

13 · PCA utiliza os rótulos?

14 · O que é SVD?

15 · Qual a relação entre PCA e SVD?

Gabarito comentado nas notas deste slide e no roteiro do professor.

Investigando a redução de dimensionalidade

Dataset Wine · load_wine() · 178 amostras · 13 características químicas

1–3	4–7	8–10	11–12
carregar, explorar, centralizar	covariância, autovalores, EVR, dois componentes	projetar, gráfico 2D, comparar com Scikit-learn	padronizado vs. não padronizado, identificados; interpretar

Entrega: repositório GitHub com notebook executado (Restart → Run All) e README.

Aula 6.11 · PCA por Álgebra Linear

Resumo da aula

- Redução de dimensionalidade
- Centralização e covariância
- Autovalores e autovetores
- Componentes principais e variância explicada
- Projeção e reconstrução
- SVD e PCA do Scikit-learn

